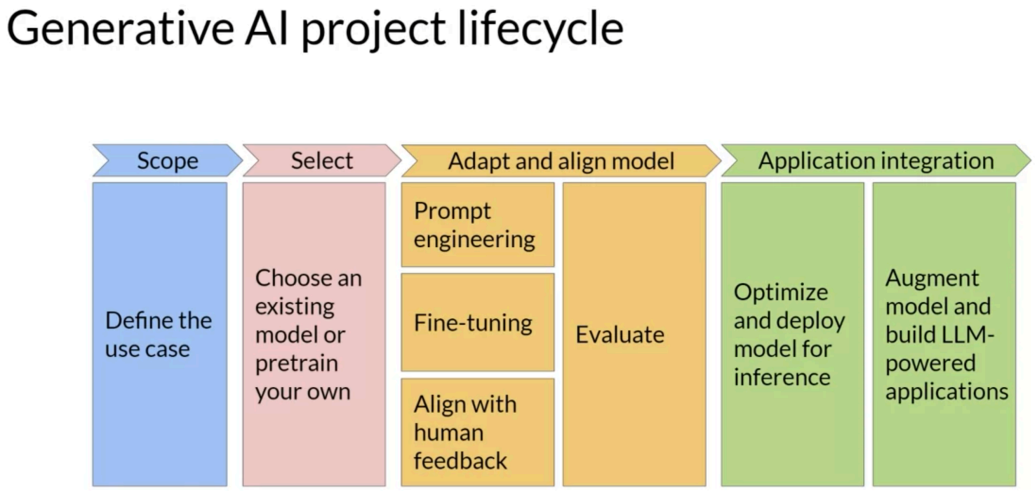
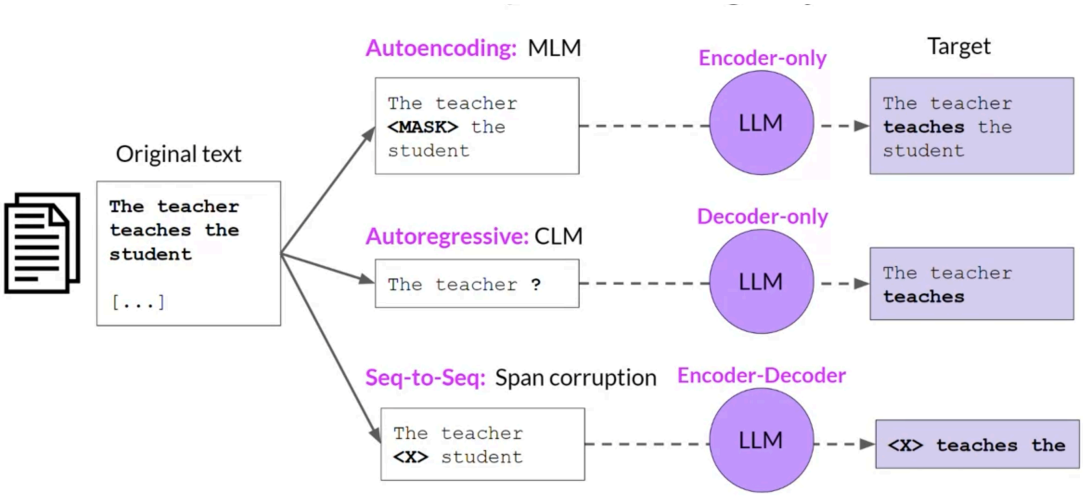
Wednesday, 21 October 2026

Pre-training LLMs & Scaling Laws



Based on the representation above we will be going through this module.

- Three Transformer Architectures and Their Pretraining Objectives



Quantization is covered in Basics of Training Optimization .

Multi GPUs efficient compute

There are 2 scenarios when train models on multiple GPUs

1 - Model parameters fit entirely on a single GPU

2 - Model parameters doesn't fit on a single GPU

For scenario 1 we use

DDP (Distributed Data Parallel)

Its a form of model training where we where we have multiple GPUs and each GPU can store the full parameters of the model,

Steps for the DDP
1- parameter distribution
    we distribute the same copy of the model to each gpu
2- Forward pass
    each copy of the model is provided with their own batch to do the forward pass and calculate the loss
3-Backprop
    Every model calculates the gradients according to the params
4- Synchronisation
    We synchronise models by taking over the average of the gradients by each model and then we update the weights according to those average gradients, this way the model is being trained and every gpu remains synchronised

For scenario 1 we use

FSDP (Fully Sharded Data Parallel)

Its a form of Model training where Models parameters dosent fit entirely on a single GPU so you have to use FSDP
- it divides the model parameters into shards
- Its distributes the models optimization state, gradients and parameters to all the available GPUs.

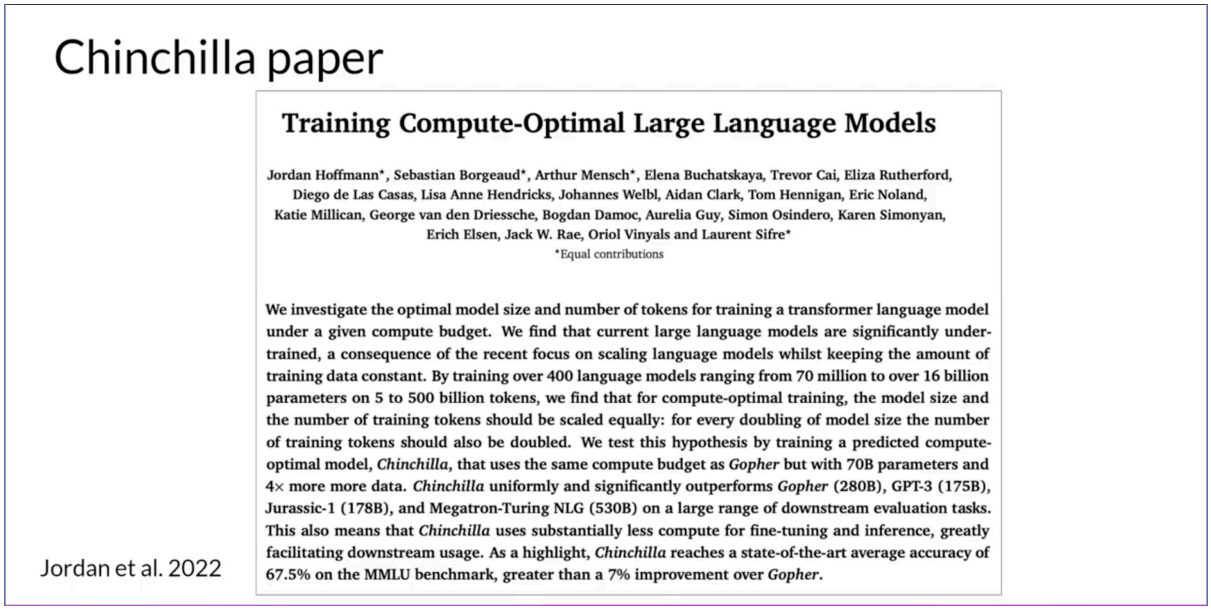
Forward pass -
    every GPU gathers the shards of a single layer and performs the forwards pass and then stores its activations and gathers another layer till we get the final output.

Backward pass -
    Reconstruct parameters as needed and calculate gradients in reverse layer order, and combine the gradient across the GPUs and distribute the reduced gradient shards.
Optimization step -
    Update the local parameter and optimizer-state shards. Repeat with the next batch.

Note - each gpu must be capable of holding all shards of a single layer while a single shard of the every other layer, plus their gradients and optimiser states (Two floating point integers per parameter)

LLM Scaling Laws

tldr -> ~ 20 tokens per param



According to Chinchilla scaling laws you should be training your model on ~ 20x the number of parameters

---

Pretraining for domain adaptation

If the vocabulary of the work you are doing or the information that you are working with is very niche you would need domain adaptation, which is to train the model on the specific data for the specific job.
For example - creating a language model to solely answer the questions on law, you would need domain adaptation.
Ex - Bloomberg gpt
It was trained on 51% financial data and 49% public data.